# ECM2433 Mock Exam Paper 5

The C Family: C, C++, and Rust

Module: ECM2433

Time allowed: 2 Hours
Total marks: 100

## Instructions

- Answer all questions.
- Marks for each part of a question are shown in brackets.
- Total marks for this paper: 100.

## 1 Question 1: C Programming, Dynamic Memory, and Preprocessor (25 marks)

### (a)

The `main` function in C can accept command-line arguments via `int argc` and `char **argv`.

(i) Explain the purpose and contents of `argc` and `argv`. [3 marks]

(ii) Write a short C loop that prints all command-line arguments supplied to the program, one per line. [2 marks]

### (b)

Allocating a 2D array dynamically requires multiple calls to `malloc`.

(i) Write a function `int **allocate_2d(int rows, int cols)` that allocates a 2D array of integers. [4 marks]

(ii) Explain the steps necessary to safely `free` this 2D array. [2 marks]

### (c)

In C, header files usually contain "include guards" (e.g., `#ifndef`, `#define`, `#endif`). Explain why include guards are necessary when structuring a program across multiple `.c` and `.h` files. [4 marks]

### (d)

A generic merge sort function uses a function pointer for comparisons:

```
1 int (*compare)(const void *p, const void *q)
```

Explain how casting is used inside the comparison function to allow this pointer to sort an array of `doubles`. [5 marks]

**(e)**

Given the following declaration:

```
1 char buffer[50] = "Exam";
```

What are the expected return values of `sizeof(buffer)` and `strlen(buffer)`? Explain the conceptual difference between the two functions. [5 marks]

## 2 Question 2: C++ STL, Streams, and Overloading (25 marks)

### (a)

Consider the following structure representing a student:

```
struct Student {
    std::string name;
    int mark;
};
```

Using the C++ Standard Template Library (STL) and a lambda expression, write a code snippet that uses `std::count_if` to count how many students in a `std::vector<Student>` scored 70 or above. [6 marks]

### (b)

Write the prototype and implementation to overload the `<<` operator for the `Student` struct, so that `std::cout << student;` prints `"Name (Mark)"`. [5 marks]

### (c)

C++ stream manipulators allow for precise output formatting. Write a C++ snippet that outputs a `double` variable `x` to exactly 3 decimal places, ensuring that trailing zeros are printed. [5 marks]

### (d)

A developer rewrites a C program in C++ but wants to call a legacy C library function `void compute_stats(int* data);`. Why must the developer wrap the C function declaration in an `extern "C"` block? How does this relate to C++ name mangling? [5 marks]

### (e)

Explain the primary benefit of using `std::unique_ptr` over raw pointers allocated with `new`. What happens to the memory when the `unique_ptr` goes out of scope? [4 marks]

## 3 Question 3: Rust Enums, Iterators, and Shared Ownership (25 marks)

### (a)

Rust's `enum` can store data within its variants. Consider the following definition:

```
enum Status {
    Pending,
    InTransit(String),
    Delivered,
}
```

Write a function `print_status(s: &Status)` that uses a `match` statement to print "Waiting" for `Pending`, "En route via [City]" for `InTransit` (where [City] is the stored string), and "Arrived" for `Delivered`. [6 marks]

### (b)

In Rust, `Rc<T>` is used for reference-counted shared ownership.

(i) Explain the difference between `Box<T>` and `Rc<T>`. [3 marks]

(ii) If you call `Rc::clone(&my_rc)`, what exactly is duplicated: the pointer, the reference count, or the heap data? [2 marks]

### (c)

Using Rust's iterator adapters (`iter`, `filter`, `map`, `collect`), write a single expression that takes a `Vec<i32>`, filters out any negative numbers, adds 10 to the remaining numbers, and collects them into a new `Vec<i32>`. [5 marks]

### (d)

Rust uses the `?` operator for concise error propagation. Given a function returning `Result<String, std::io::E` explain exactly what happens when `File::open("data.txt")?` encounters an error versus when it succeeds. [5 marks]

### (e)

Rust's borrow checker enforces strict mutability rules. Explain why the following code is rejected by the compiler:

```
let mut text = String::from("Hello");
let r1 = &mut text;
let r2 = &mut text;
println!("{} {}", r1, r2);
```

[4 marks]

## 4 Question 4: Language Comparison and Paradigms (25 marks)

### (a) Null Pointers vs. Option

In C and C++, missing or invalid data is often represented using `NULL` or `nullptr`. In Rust, it is represented using the `Option<T>` enum. Explain how Rust's approach forces the programmer to handle the "missing" case at compile time, preventing accidental null pointer dereferences. [6 marks]

### (b) Concurrency and Data Races

To safely share a hash map across multiple threads:

(i) In C++, a programmer might use a `std::mutex`. Explain the risk if the programmer forgets to lock the mutex before modifying the map. [3 marks]

(ii) In Rust, the standard approach is to wrap the map in `Arc<Mutex<HashMap>>`. Explain how Rust's type system physically prevents a thread from modifying the map without first locking the mutex. [4 marks]

### (c) Polymorphism

Compare how dynamic dispatch (polymorphism) is achieved in C++ versus Rust.

(i) Explain the role of `virtual` functions and base class pointers in C++. [3 marks]

(ii) Explain the role of **Traits** and `Box<dyn Trait>` in Rust. [3 marks]

### (d) Manual vs. Automatic Iteration

When iterating over an array of size $N$:

- C approach: `for (int i = 0; i < N; i++) { printf("%d", arr[i]); }`
- Rust approach: `for x in &arr { println!("{}", x); }`

Give two distinct safety benefits of using the Rust iterator approach over the C manual indexing approach. [6 marks]

**End of Paper**